NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007/CE1007/CZ1007

Queue Data Structure



Presented by Dr Owen Noel Newton Fernando

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In this lecture, we'll explore queues, a fundamental linear data structure used throughout computing. We'll examine their FIFO behaviour and learn how they can be implemented using linked nodes and a LinkedList class.



Objectives

- Introduce the Queue data structure and its FIFO behaviour.
- Explain the roles of the front and rear in a Queue.
- Discuss the core operations performed on a Queue.
- Explain how a Queue can be implemented using a linked list.
- Demonstrate how enqueue and dequeue modify the Queue structure.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The objectives of this lecture are to develop a clear understanding of how a Queue behaves and how it can be implemented. By the end, you should recognise a Queue as a precise way of controlling how data enters and leaves in FIFO order.

We'll examine how the front and rear references support this behaviour, then trace the changes produced by enqueue and dequeue. We'll pay particular attention to empty and single-node Queues, as these boundary cases often reveal implementation errors.

You'll also compare direct linked-node code with a Queue built using an existing LinkedList class. This will help you understand the underlying node mechanics and compare how the two implementations support the same Queue operations.

Learning Outcome

- Explain the FIFO behaviour of a Queue.
- Identify the roles of the front and rear in a Queue.
- Apply the core Queue operations to manipulate Queue elements.
- Implement a Queue using a linked list in Python.
- Trace the changes caused by enqueue and dequeue operations.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

The intended learning outcomes describe what you should be able to explain and apply after completing this lecture. You should understand FIFO behaviour and identify how the front and rear control where elements leave and enter a Queue.

You should also be able to apply the core operations and trace how enqueue() and dequeue() change the stored nodes. When reading Python code, follow how the front, rear and size are updated, particularly when the final node is removed.

Beyond tracing the code, you should be able to implement a Queue using a linked list. You should also understand how the front and rear references provide direct access to the two ends of the Queue without unnecessary traversal of the linked nodes.

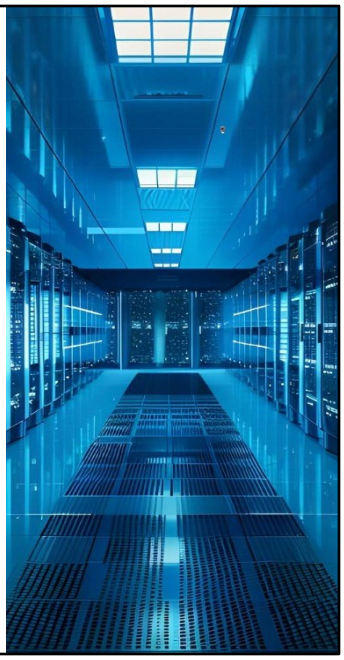
Contents



- Introduction to Queues
- Implementing a Queue Using Linked Nodes
- Working Example Using Linked Nodes
- Implementing a Queue Using a LinkedList Class
- Working Example Using a LinkedList Class
- Queue Applications
- Reversing a Queue Using a Stack
- Conclusion

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted



To give you a clear sense of today's lecture, we'll begin by introducing Queues and examining how FIFO controls access to their elements. This foundation will explain why insertion and removal take place at opposite ends.

We'll then implement a Queue directly using linked nodes and examine a complete working example. After that, we'll create the same Queue using a LinkedList class, allowing us to compare the two implementation approaches.

The final sections connect Queues to computing tasks such as scheduling, buffering and graph traversal. We will then combine a Queue with a Stack to see how their different access orders can be used to reverse a sequence, before concluding with the main ideas from the lecture.

Introduction to Queues

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We'll begin by developing a clear understanding of what a Queue is and how it controls access to stored elements. This section introduces the roles of the front and rear, together with the FIFO principle that determines the order in which elements are processed.

What is a Queue?

- A queue is a linear data structure that stores elements in a sequence.
- Access to elements is restricted to the two ends of the sequence.
- The front identifies the element that is available for removal.
- The rear identifies where a new element is inserted.
- Elements cannot normally be inserted or removed from arbitrary positions.
- A queue can be implemented using a list or linked list.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Think about the waiting line shown here: people join at one end and are served from the other. A Queue follows the same basic arrangement, storing elements in sequence while restricting where they can enter and leave.

The front identifies the element that has waited longest and is therefore available for removal next. New elements join at the rear, so they cannot move ahead of elements already waiting.

These access rules remain the same whether the Queue is implemented using an array-based list or linked nodes. In this lecture, we'll focus on linked storage, where front and rear references provide direct access to both ends.

Understanding FIFO Behaviour

- A queue follows the First In, First Out (FIFO) principle.
- Elements are processed in the order in which they arrive.
- The element that has been in the queue longest is removed first.
- New elements join at the rear, behind those already waiting.
- Removing the front element makes the next waiting element the new front.
- FIFO therefore maintains a fair and predictable processing order.



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

FIFO tells us exactly which element should be processed next: the one that entered the Queue first. An element arriving later must remain behind those that are already waiting, regardless of its value.

The enqueue() operation places each new element at the rear, preserving the order established by earlier arrivals. When dequeue() removes the front element, the element immediately behind it becomes the new front.

This produces a fair and predictable processing sequence without rearranging the stored elements. As we later trace the Python code, checking the arrival order will help you confirm whether the implementation still follows FIFO correctly.

Why Do We Use Queues?

- Queues are useful when elements need to be processed in the order they arrive.
- They provide a systematic way to manage waiting tasks, requests, or data.
- New elements can be added without changing the order of elements already waiting.
- The FIFO principle ensures that earlier elements are processed before later ones.
- Queues are commonly used for task scheduling, request processing, buffering, and breadth-first traversal.
- Queues can manage changing sequences of data while preserving their processing order.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Consider a program receiving tasks faster than it can process them; those tasks need somewhere organised to wait. A Queue provides that waiting space while preserving the order in which each task arrived.

New requests can join at the rear without disturbing older requests already waiting at the front. This makes Queues suitable for task scheduling, buffering, event handling and breadth-first traversal of graphs.

These applications may involve many insertions and removals. By maintaining direct front and rear references, a linked Queue can add and remove elements without repeatedly traversing the entire sequence.

Implementing a Queue Using Linked Nodes

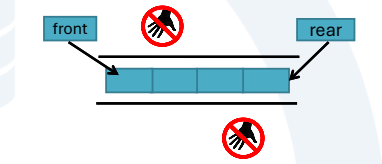
© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

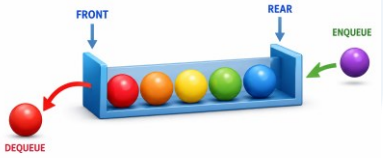
We'll now move from the Queue concept to a direct implementation using linked nodes. We'll examine how the front and rear references manage both ends and how each operation updates the links while preserving FIFO order.

Implementing a Queue using Linked Nodes

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0
```





The rear reference provides direct access to the end of the Queue, allowing new elements to be enqueued without traversing the linked nodes.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At the foundation of this implementation is the Node class, which stores some data and a reference to the next node. Each node therefore holds one Queue element and provides the link needed to reach the following node.

When a Queue is first created, both front and rear are set to None because it contains no nodes. The size attribute also begins at zero and will change whenever an element enters or leaves the Queue.

Once nodes are added, front refers to the first node in the Queue, while rear refers to the last node. A new node is linked after the current rear and then becomes the new rear.

Core Queue operations

- **getFront():** Inspect the element at the front of the Queue without removing it.
- **enqueue():** Add an element at the rear of the Queue.
- **dequeue():** Remove and return the element at the front of the Queue.
- **isEmpty():** Check whether the Queue is empty.
- **getSize():** Returns the number of elements stored in the Queue.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A Queue is used through a small set of operations rather than by accessing its linked nodes directly. This keeps the internal representation hidden and allows callers to focus on what each operation does.

enqueue() and dequeue() modify the structure by adding at the rear and removing from the front. In contrast, getFront() lets us inspect the next available element without changing the Queue.

The other two methods describe its current state: isEmpty() checks whether any elements remain, while getSize() reports their number. As we examine the implementation, watch how these methods work together to preserve FIFO order and handle an empty Queue safely.

Core Queue operations: getFront()

- Inspect the element at the front of the Queue without removing it.

```
def getFront(self):
        if self.isEmpty():
            raise IndexError("Peek from empty queue")
        return self.front.data
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Sometimes we need to check which element will leave the Queue next without actually removing it. The getFront() method provides this view while leaving the linked structure completely unchanged.

Before accessing any node, the method calls isEmpty() to confirm that a front element exists. If the Queue is empty, it raises an exception rather than returning a value that could be misunderstood.

When elements are present, self.front refers to the first node in the linked sequence. Returning self.front.data gives the caller its stored value without changing the front, rear, size or links between nodes.

Core Queue operations: enqueue()

- The rear reference keeps track of the last node in the Queue.
- Without a rear reference, we would need to traverse the linked nodes to find the last node before each enqueue operation.




```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The purpose of enqueue() is to place a new element at the rear without disturbing the order already established in the Queue. It begins by creating a node to hold the incoming data, with its next reference initially set to None.

What happens next depends on whether the Queue already contains any nodes. If it is empty, the new node becomes the front; otherwise, the current rear node must be linked to it.

The rear reference then moves to the new node because it is now the final element. Finally, the size increases, keeping the recorded number of elements consistent with the linked structure.

enqueue(): Empty Queue

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: **Create a new node with the data.**
Step 2: If the queue is empty, set the front to the new node.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: Make the new node the rear.
Step 5: Increase the size count

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

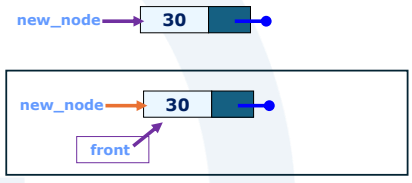
The first step is simply to create a new node containing the value 30. At this moment, new_node is a local reference, and the node's next field is None because it is not linked to anything yet.

Notice that creating the node has not actually placed it inside the Queue. The Queue remains empty because neither front nor rear refers to this new object at this stage.

The method therefore checks isEmpty() to decide which insertion path should be followed. Since this is the first node, the next statement will connect the Queue to it through the front reference.

enqueue(): Empty Queue (cont.)

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: Create a new node with the data.
Step 2: **If the queue is empty, set the front to the new node**.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: Make the new node the rear.
Step 5: Increase the size count

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

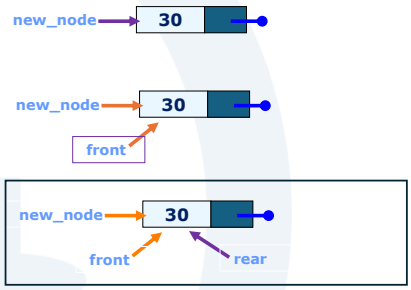
Because the Queue is empty, the method follows the first branch and assigns new_node to self.front. The Queue now has its first structural reference to the node containing 30.

At this stage, both new_node and front refer to the same node, as the diagram shows. Its next field remains None, which is correct because there is no second element to link to.

The insertion is not quite complete, however, because rear has not yet been updated. Since this node is currently both the first and last element, the rear reference must point to it as well.

enqueue(): Empty Queue (cont.)

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: Create a new node with the data.
Step 2: If the queue is empty, set the front to the new node.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: **Make the new node the rear.**
Step 5: Increase the size count

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

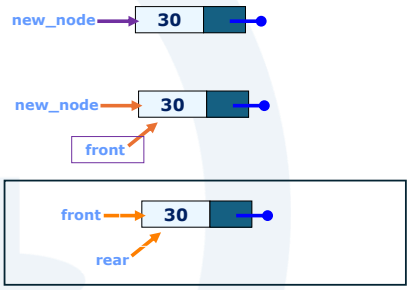
With only one node in the Queue, front and rear must both refer to that same node. The assignment self.rear = new_node establishes this arrangement for the node containing 30.

No additional link is required because the node is already at the end of the Queue. Its next reference therefore remains None, showing that no element follows it.

The structural references are now correct, but the recorded size is still zero at this point in the method. The remaining statement will increase size to one, bringing the counter into agreement with the linked structure.

enqueue(): Empty Queue (cont.)

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: Create a new node with the data.
Step 2: If the queue is empty, set the front to the new node.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: Make the new node the rear.
Step 5: **Increase the size count**

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

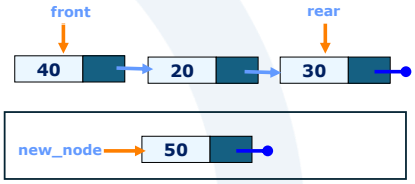
Increasing size from zero to one completes the first enqueue operation. The stored count now agrees with the single node in the linked structure.

Both front and rear refer to node 30 because it is simultaneously the first and last element. Its next reference remains None, correctly marking the end of the Queue.

This one-element state forms the boundary between an empty Queue and a Queue containing several nodes. A later enqueue will retain the same front, connect another node at the back and move the rear reference.

enqueue(): Queue contains many Nodes

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: **Create a new node with the data.**
Step 2: If the queue is empty, set the front to the new node.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: Make the new node the rear.
Step 5: Increase the size count

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

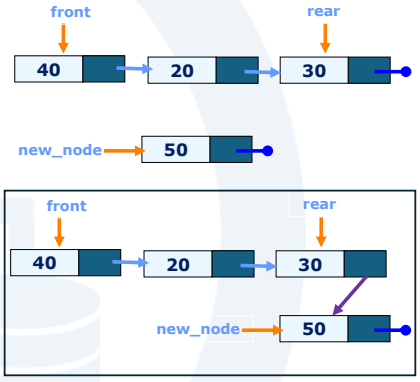
This example begins with a non-empty Queue containing 40, 20 and 30, with front referring to 40 and rear referring to 30. We now create a separate node containing the new value 50.

At this moment, the existing Queue remains unchanged because the new node has not yet been linked to it. Since isEmpty() returns False, the method skips the branch that updates front.

Instead, it uses the current rear node containing 30 to extend the linked sequence. The next step will connect that node's next reference to new_node, placing 50 after all existing elements.

enqueue(): Queue contains many Nodes (cont.)

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



Step 1: Create a new node with the data.
Step 2: If the queue is empty, set the front to the new node.
Step 3: **Otherwise, link the current rear node to the new node**.
Step 4: Make the new node the rear.
Step 5: Increase the size count

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

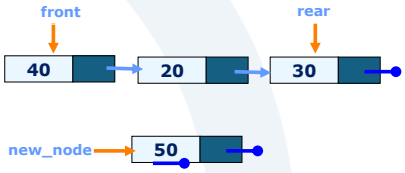
The rear reference now moves from node 30 to the newly inserted node containing 50. Increasing size completes the operation, so the Queue now records all four elements correctly.

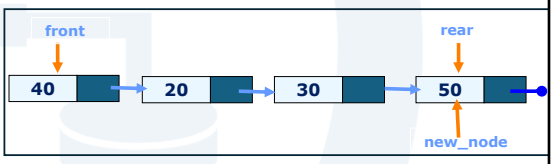
Starting from front, we can follow the links through 40, 20 and 30 until reaching 50 at the rear. The original arrival order is preserved, and 40 remains the next element available for removal.

new_node is a local reference inside enqueue(). When the method completes, this local reference goes out of scope, but the node remains part of the Queue because it is referenced through the linked structure and rear.

enqueue(): Queue contains many Nodes (cont.)

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```





Step 1: Create a new node with the data.
Step 2: If the queue is empty, set the front to the new node.
Step 3: Otherwise, link the current rear node to the new node.
Step 4: **Make the new node the rear.**
Step 5: **Increase the size count**

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Core Queue operations: dequeue()

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Unlike enqueue(), the dequeue() operation works at the front of the Queue and removes the element that arrived first. It begins by checking for an empty Queue, since there would be no valid node to remove.

For a non-empty Queue, dequeued_node temporarily saves the current front node and its data. The front reference then advances to the next node, removing the saved node from the Queue's active sequence.

If no next node exists, rear must also become None because the Queue is now empty. The method then decreases size and returns the data stored in dequeued_node.

dequeue(): Empty Queue

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: **If the queue is empty, raise an error.**
- Step 2: Store the current front node in dequeued_node.
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This first case shows what happens when dequeue() is called on an empty Queue. Since front is None, there is no node available to remove and no data that can be returned.

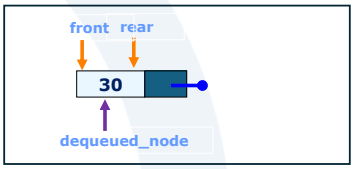
The method detects this condition through self.isEmpty() before attempting to access self.front. It then raises an IndexError with a clear message explaining that dequeue cannot operate on an empty Queue.

Execution stops at that point, so none of the remaining statements in the method are performed. The Queue stays empty, its references remain unchanged, and the invalid operation is reported directly to the caller.

dequeue(): Queue contains Single Node

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: **Store the current front node in dequeued_node.**
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This example begins with a Queue containing only one node, which stores the value 30. Because it is the only node, both front and rear refer to the same object.

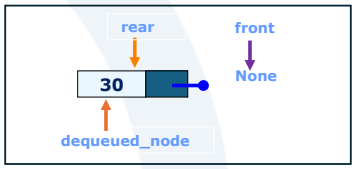
The assignment dequeued_node = self.front creates a temporary local reference to that node. At this moment, front, rear and dequeued_node all point to node 30, but each reference serves a different purpose.

dequeued_node preserves access to the value that must eventually be returned. We can therefore update front safely, knowing that the removed node's data will remain available through this local reference.

dequeue(): Queue contains Single Node (cont.)

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: **Move the front reference to the next node.**
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The statement self.front = self.front.next moves the front reference away from the node containing 30. Since this is the only node, its next reference is None.

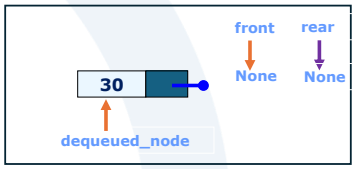
Following that reference therefore makes self.front become None. From the front of the Queue, there is now no node available for access or removal.

Notice that this statement does not delete the node directly or change the data stored inside it. It simply changes where front points, which removes node 30 from the active linked sequence while dequeued_node continues to preserve access to it.

dequeue(): Queue contains Single Node (cont.)

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: Move the front reference to the next node.
- Step 4: **If the new front is None, set the rear to None.**
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At this stage, front has already moved to None because node 30 has no next node. However, rear still refers to node 30, so the Queue has not yet been fully updated to represent the empty state.

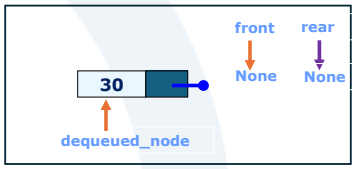
The condition self.front is None tells us that the node just removed was the only node in the Queue. Since node 30 was both the front and the rear, rear must also be updated when that node is removed.

Therefore, self.rear = None clears the remaining reference to node 30. Now both front and rear are None, correctly representing an empty Queue, before the size is decreased and the removed data is returned.

dequeue(): Queue contains Single Node (cont.)

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: **Decrease the size count.**
- Step 6: **Return the data of dequeued_node**

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

With both front and rear set to None, the linked structure now correctly represents an empty Queue. Decreasing size from one to zero brings the stored count into agreement with those references.

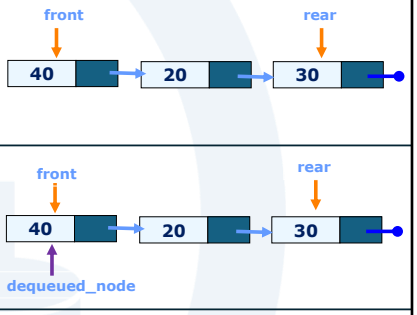
The method then returns dequeued_node.data, so the caller receives 30 rather than the internal node object. Keeping the node hidden prevents outside code from changing the Queue's links.

This completes the one-node boundary case: a single dequeue() has taken the Queue from one element to none. The next example contrasts this with removing the front while other nodes remain.

dequeue(): Queue contains many Nodes

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: **Store the current front node in dequeued_node.**
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This example shows how dequeue() removes the front node from a Queue containing several nodes. The values are arranged as 40, 20 and 30, so FIFO requires node 40 to leave first.

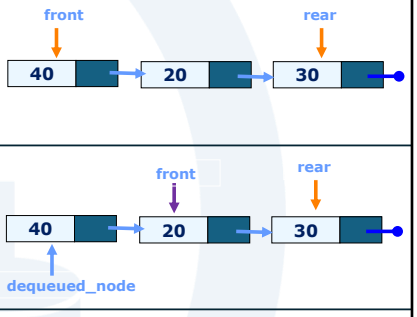
Before changing the structure, dequeued_node is assigned the same node currently referenced by front. This local reference preserves access to node 40 and its data while the Queue's own front reference is updated.

The other nodes remain linked in their existing order, and rear continues to identify node 30. Unlike the one-node case, removing node 40 will leave nodes 20 and 30 in the Queue, so the Queue will remain non-empty.

dequeue(): Queue contains many Nodes (cont.)

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```



- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: **Move the front reference to the next node.**
- Step 4: If the new front is None, set the rear to None.
- Step 5: Decrease the size count.
- Step 6: Return the data of dequeued_node

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Here the Queue contains several nodes, so advancing front behaves differently from the one-node case. The assignment self.front = self.front.next bypasses node 40 and makes node 20 the new front.

The local reference dequeued_node still points to node 40, preserving its data until the method returns it. However, node 40 is no longer part of the active Queue sequence.

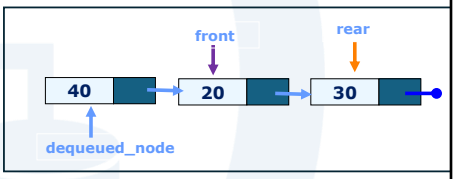
Because front now refers to node 20 rather than None, the condition is false and rear is not cleared. It correctly remains at node 30, while nodes 20 and 30 preserve their original order.

dequeue(): Queue contains many Nodes (cont.)

- Remove the node at the front of the Queue

```
def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data
```





- Step 1: If the queue is empty, raise an error.
- Step 2: Store the current front node in dequeued_node.
- Step 3: Move the front reference to the next node.
- Step 4: If the new front is None, set the rear to None.
- Step 5: **Decrease the size count.**
- Step 6: **Return the data of dequeued_node**

After the method returns, if no other reference to node 40 exists, the node becomes unreachable and is eligible for automatic memory reclamation by Python.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The final statements complete the removal of node 40 from the Queue. self.size decreases to reflect the two remaining nodes, and the method returns the data saved through dequeued_node.

At this stage, front refers to node 20 and rear still refers to node 30. These two nodes remain linked in their original order, so node 20 will be removed next under FIFO behaviour.

When the method returns, the local reference dequeued_node goes out of scope. If no other reference to node 40 exists, the node becomes unreachable and Python can automatically reclaim its memory, while the returned data remains available to the caller.

Core Queue operations: isEmpty()

- Check whether the queue is empty

```
def isEmpty(self):
        return self.front is None
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

How can the Queue determine whether it contains any elements without examining every node? It only needs to inspect front, because every non-empty linked Queue must have a first node.

The expression self.front is None produces a Boolean value directly. It returns True when the Queue is empty and False when front refers to a node.

Other methods use this check before accessing or removing the front element, helping them handle an empty Queue safely. We could instead test whether size is zero, but checking front expresses emptiness through the actual linked structure and requires no traversal.

Core Queue operations: getSize()

- Return the number of elements in the Queue.

```
def getSize(self):
        return self.size
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Rather than counting the nodes each time, getSize() returns the value already stored in self.size. This gives the caller the current number of Queue elements without traversing the linked structure.

The counter increases whenever enqueue() adds a node and decreases whenever dequeue() removes one. As a result, retrieving the Queue's size takes the same amount of time regardless of how many elements it contains.

There is a trade-off, however: every structural operation must update the counter correctly. A missed increment or decrement would make size disagree with the actual nodes, which is why we tracked it carefully throughout the earlier traces.

Working Example Using Linked Nodes

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We have explored the Queue methods individually, and we can now see how they work together in a complete implementation. As we follow the example, watch how front, rear and size remain consistent across each operation. Try to predict the Queue's state and the printed output before each result is revealed.

Working Example- 01

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def isEmpty(self):
        return self.front is None

    def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1

    def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        dequeued_node = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self.size -= 1
        return dequeued_node.data

    def getFront(self):
        if self.isEmpty():
            raise IndexError("Peek from empty queue")
        return self.front.data

    def getSize(self):
        return self.size
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

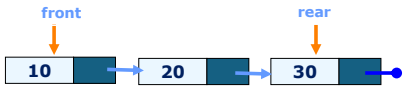
Here we can see the complete linked-node implementation rather than isolated methods. The Node class provides the data and link, while the Queue constructor establishes the empty state using front, rear and size.

Every method works with these same three attributes, so consistency across the class matters. enqueue() handles the first node separately, while dequeue() includes the corresponding check for removing the final node.

The inspection methods expose useful information without revealing the internal node objects. Together, these methods preserve FIFO behaviour and allow the main Queue operations to run without traversing the linked sequence.

Working Example- 01 (cont.)

```
40 if __name__ == "__main__":
41
42     queue = Queue()
43
44     queue.enqueue(10)
45     queue.enqueue(20)
46     queue.enqueue(30)
47
48     print("Front element before dequeue:", queue.getFront())
49
50     print("Size before dequeue:", queue.getSize())
51
52     print("Dequeued element:", queue.dequeue())
53
54     print("Front element after dequeue:", queue.getFront())
55
56     print("Size after dequeue:", queue.getSize())
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

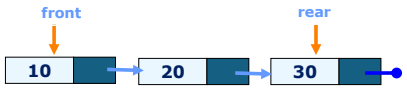
The test program begins by creating an empty Queue and then enqueues 10, 20 and 30 in that order. This gives us a short sequence that is easy to trace while still demonstrating FIFO behaviour.

The diagram shows 10 at the front because it arrived first, while 30 occupies the rear because it arrived last. At this stage, all three nodes are linked and the Queue's size should match the number of stored elements.

The print statements then apply the main Queue operations in sequence, beginning with getFront() and getSize(). They continue with dequeue() before checking the updated front and size, allowing us to observe how one removal changes the Queue's state.

Working Example- 01 (cont.)

```
40 if __name__ == "__main__":
41
42     queue = Queue()
43
44     queue.enqueue(10)
45     queue.enqueue(20)
46     queue.enqueue(30)
47
48     print("Front element before dequeue:", queue.getFront()) # 10
49
50     print("Size before dequeue:", queue.getSize()) # 3
51
52     print("Dequeued element:", queue.dequeue()) # 10
53
54     print("Front element after dequeue:", queue.getFront()) # 20
55
56     print("Size after dequeue:", queue.getSize()) # 2
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The outputs beside lines 48 to 56 show what each Queue operation returns. Line 48 reports 10 as the front element, and line 50 confirms that three elements are stored before removal.

Line 52 calls dequeue(), which returns 10 because it was the first value added. This confirms that the Queue follows arrival order rather than selecting elements according to their numeric values.

After the removal, line 54 reports 20 as the new front element. Finally, line 56 returns a size of two, matching the remaining nodes containing 20 and 30.

Implementing a Queue Using a LinkedList Class

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

So far, our Queue has managed each linked node directly, making every reference change visible. We will now build the same Queue behaviour by reusing an existing LinkedList class, with the Queue delegating its storage operations through self.ll. This approach reduces duplicated code while still preserving the correct mapping between the Queue's front and rear and the linked list's head and tail.

Same Queue Interface, New Storage

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
        self.tail = None
```

```
class Queue:
    def __init__(self):
        self.ll = LinkedList()
```

Implementing a Queue Using a LinkedList Class

- The `Queue` uses a `LinkedList` for storage.
- `self.ll` creates the underlying linked list.
- `head` represents the front of the `Queue`.
- `tail` represents the rear of the `Queue`.
- `size` tracks the number of nodes.
- `Queue` operations are performed through the `LinkedList`.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Here, the Queue does not manage its own nodes directly. Its constructor creates a LinkedList object in self.ll, and that object provides all the underlying storage.

The list's head corresponds to the Queue's front because the oldest element must be removed from the beginning. Its tail corresponds to the rear, where each newly enqueued element is added.

The Queue can also use the LinkedList's size field instead of maintaining a second counter. This keeps the responsibilities separate: the LinkedList handles nodes and references, while the Queue exposes only the operations needed to preserve FIFO behaviour.

Core Queue operations

- **getFront():** Inspect the element at the front of the Queue without removing it.
- **enqueue():** Add an element at the rear of the Queue.
- **dequeue():** Remove and return the element at the front of the Queue.
- **isEmpty():** Check whether the Queue is empty.
- **getSize():** Returns the number of elements stored in the Queue.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

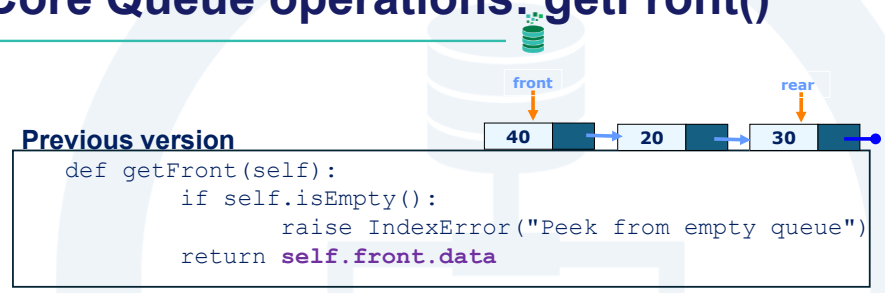


Changing the storage structure does not change what users can ask the Queue to do. The same five operations remain available, so existing client code can continue using the class in exactly the same way.

What changes is the work performed inside each method. Instead of updating Queue-owned node references, the methods will call suitable operations on the LinkedList stored in self.ll.

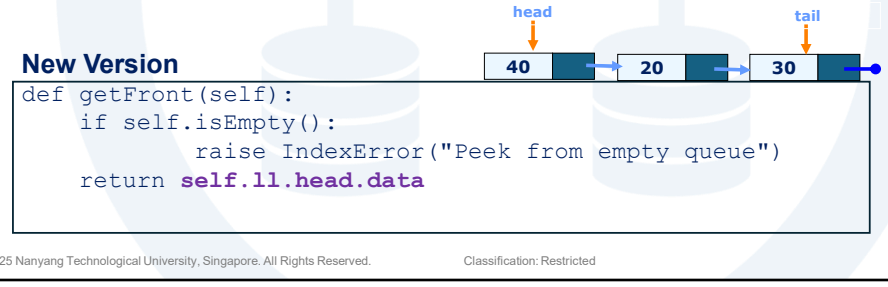
The list's head represents the Queue's front, while its tail identifies the rear. We will examine these mappings one method at a time, showing how the familiar Queue interface can be preserved while the LinkedList handles the underlying nodes.

Core Queue operations: getFront()



Previous version

```
def getFront(self):
        if self.isEmpty():
                raise IndexError("Peek from empty queue")
        return self.front.data
```



New Version

```
def getFront(self):
    if self.isEmpty():
            raise IndexError("Peek from empty queue")
    return self.ll.head.data
```

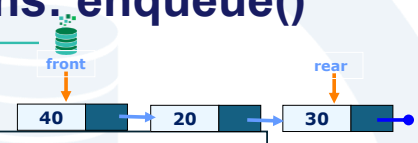
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Compare the final return statement in these two versions of getFront(). The direct implementation reads from self.front, whereas the LinkedList-based version follows self.ll.head to reach the same first element.

The empty check is still required because an empty LinkedList has no head node whose data can be accessed. If the Queue is empty, the method raises the same Queue-level exception as before.

From the caller's perspective, nothing has changed: getFront() still returns 40 without removing it. Only the internal route to that value differs because the LinkedList now owns and manages the nodes.

Core Queue operations: enqueue()



Previous version

```
def enqueue(self, data):
        new_node = Node(data)
        if self.isEmpty():
            self.front = new_node
        else:
            self.rear.next = new_node
        self.rear = new_node
        self.size += 1
```



New version

```
def enqueue(self, data):
        self.ll.insert_node(self.ll.size, data)
```

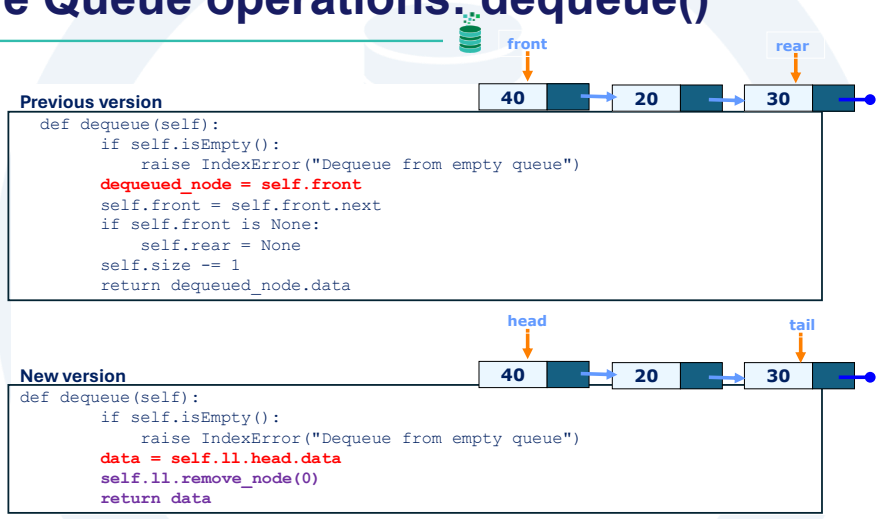
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The contrast is much clearer for enqueue(). Previously, the Queue created a node, handled the empty case, updated the rear link and increased its own size.

The new version delegates that work to self.ll.insert_node(). Passing self.ll.size as the index places the new data immediately after the current last element, which corresponds to joining the Queue at the rear.

The LinkedList then manages the node creation, tail reference and size update internally. If insertion at the tail uses the stored tail reference directly, the Queue can still enqueue without traversing the existing nodes.

Core Queue operations: dequeue()



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The LinkedList-based dequeue() begins with the same safety check as the direct version. If the Queue is empty, it raises an exception before attempting to access the list's head.

For a valid removal, the method first saves self.ll.head.data in the local variable data. It then calls remove_node(0), asking the LinkedList to remove the first node in its sequence.

The LinkedList is responsible for updating its head, tail and size, including the one-node boundary case. The Queue simply returns the saved data, so callers observe exactly the same FIFO behaviour despite the shorter implementation.

Core Queue operations: isEmpty()

Previous version

```
def isEmpty(self):
    return self.front is None
```



New Version

```
def isEmpty(self):
    return self.ll.size == 0
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The meaning of isEmpty() has not changed, but the source of its answer has. Previously, the Queue checked whether its own front reference was None.

In the new design, the LinkedList owns the structural state, so the method tests whether self.ll.size is zero. A zero size means that the list contains no nodes and the Queue therefore contains no elements.

All Queue methods must now obtain storage information consistently through self.ll. Mixing the old Queue attributes with the LinkedList attributes would create two competing descriptions of the same structure and could produce incorrect results.

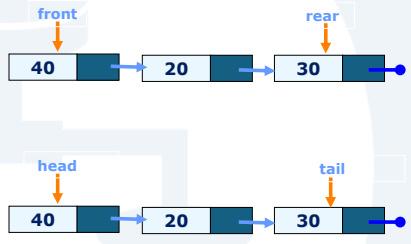
Core Queue operations: getSize()

Previous version

```
def getSize(self):
        return self.size
```

New Version

```
def getSize(self):
        return self.ll.size
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

The change to getSize() is small but follows the same design principle as isEmpty(). Instead of returning a counter owned by the Queue, the method returns self.ll.size.

The LinkedList updates this value whenever it inserts or removes a node. There is therefore no need to traverse the nodes or maintain a second size field inside the Queue.

For the caller, queue.getSize() still reports the number of stored elements exactly as before. Internally, keeping a single counter avoids inconsistent state and leaves the LinkedList responsible for all storage-related information.

Working Example Using a LinkedList Class

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We will now combine the LinkedList and Queue classes in a complete working program. Although the LinkedList supports operations at different positions, the Queue will use only insertion at the end and removal from the beginning to preserve FIFO behaviour. We will then run the familiar Queue operations and check that this implementation produces the same results as the direct linked-node version.

Working Example – 02

```
1   class Node:
2       def __init__(self, data):
3           self.data = data
4           self.next = None
5
6   class LinkedList:
7       def __init__(self):
8           self.size = 0
9           self.head = None
10          self.tail = None
11
12      def find_node(self, index):
13          if index < 0 or index >= self.size:
14              return None
15          temp = self.head
16          for _ in range(index):
17              temp = temp.next
18          return temp
19
20      def remove_node(self, index):
21          if index < 0 or index >= self.size:
22              return -1
23          if index == 0:
24              self.head = self.head.next
25              if self.size == 1:
26                  self.tail = None
27          else:
28              prev = self.find_node(index - 1)
29              prev.next = prev.next.next
30              if index == self.size - 1:
31                  self.tail = prev
32          self.size -= 1
33          return 0
```

```
34      def insert_node(self, index, value):
35          if index < 0 or index > self.size:
36              return -1
37          new_node = Node(value)
38          if index == 0:
39              new_node.next = self.head
40              self.head = new_node
41              if self.size == 0:
42                  self.tail = new_node
43          elif index == self.size:
44              self.tail.next = new_node
45              self.tail = new_node
46          else:
47              prev = self.find_node(index - 1)
48              new_node.next = prev.next
49              prev.next = new_node
50          self.size += 1
51          return 0
```

©

Classification: Restricted

This code provides the general-purpose LinkedList that will sit underneath the Queue. It maintains head, tail and size, while find_node() supports operations that need to locate an interior position.

The insertion and removal methods handle several cases because the LinkedList is not limited to Queue behaviour. They can update the beginning, the end or a position within the list while keeping the references and size consistent.

Our Queue uses only two of these paths: insertion at index size and removal at index zero. Those operations work directly through tail and head, so no traversal is needed for normal enqueue and dequeue calls.

Working Example – 02 (cont.)

```
52 class Queue:
53     def __init__(self):
54         self.ll = LinkedList()
55
56     def enqueue(self, data):
57         self.ll.insert_node(self.ll.size, data)
58
59     def dequeue(self):
60         if self.isEmpty():
61                 raise IndexError("Dequeue from empty queue")
62         data = self.ll.head.data
63         self.ll.remove_node(0)
64         return data
65
66     def getFront(self):
67         if self.isEmpty():
68             raise IndexError("Empty queue")
69         return self.ll.head.data
70
71     def getSize(self):
72         return self.ll.size
73
74     def isEmpty(self):
75         return self.ll.size == 0
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

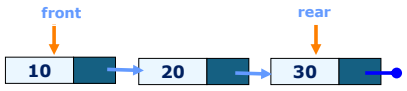
The Queue class is now much shorter because it delegates every storage operation to self.ll. Its constructor creates the LinkedList that will hold all Queue elements.

enqueue() inserts at the current list size, placing new data at the tail, while dequeue() saves the head data before removing index zero. getFront() reads from that same head without changing the list.

The remaining methods obtain the size and empty state directly from the LinkedList's counter. Because every operation uses self.ll, the class has one consistent source of information while still presenting a familiar FIFO interface to callers.

Working Example- 02 (cont.)

```
76 if __name__ == "__main__":
77
78     queue = Queue()
79
80     queue.enqueue(10)
81     queue.enqueue(20)
82     queue.enqueue(30)
83
84     print("Front element before dequeue:", queue.getFront())
85
86     print("Size before dequeue:", queue.getSize())
87
88     print("Dequeued element:", queue.dequeue())
89
90     print("Front element after dequeue:", queue.getFront())
91
92     print("Size after dequeue:", queue.getSize())
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

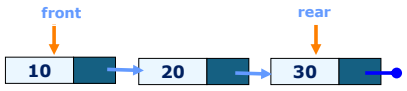
To compare the two implementations fairly, this program repeats the same sequence of operations used earlier. It creates a Queue and then enqueues 10, 20 and 30 in that order.

Although the Queue now stores these values through a LinkedList, the abstract arrangement is unchanged. The value 10 is at the front, 30 is at the rear, and three elements are waiting before removal.

The print statements inspect the front and size, perform one dequeue(), and then inspect the updated state. Before viewing the outputs, predict whether this implementation will produce exactly the same results as the direct linked-node version.

Working Example- 02 (cont.)

```
76 if __name__ == "__main__":
77
78     queue = Queue()
79
80     queue.enqueue(10)
81     queue.enqueue(20)
82     queue.enqueue(30)
83
84     print("Front element before dequeue:", queue.getFront()) # 10
85
86     print("Size before dequeue:", queue.getSize()) # 3
87
88     print("Dequeued element:", queue.dequeue()) # 10
89
90     print("Front element after dequeue:", queue.getFront()) # 20
91
92     print("Size after dequeue:", queue.getSize()) # 2
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The comments beside lines 85 to 93 show the output produced by each Queue operation. Before any removal, getFront() returns 10 because it was enqueued first, while getSize() confirms that three elements are stored.

Line 89 then calls dequeue(), which removes and returns the value 10 from the front. Inside the LinkedList, the head reference advances to the node containing 20, while the tail remains at node 30.

The final two calls allow us to check that the Queue's state was updated correctly. getFront() now returns 20 and getSize() returns two, matching the remaining sequence of 20 followed by 30.

Queue Applications

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Queues are widely used in computing whenever items must wait temporarily and be processed in an organised order. We will examine several applications where the enqueue-and-dequeue pattern appears clearly, while recognising that some systems also introduce priorities or other scheduling rules. For each example, consider what enters the Queue, what leaves, and how the processing order is determined.

Queues in computer science

- **Operating systems:**
  - queue of print jobs to send to the printer
  - queue of programs / processes to be run
  - queue of network data packets to send
- **Programming:**
  - modeling a line of customers or clients
  - storing a queue of computations to be performed in order
- **Real world examples:**
  - people on an escalator or waiting in a line
  - cars at a gas station

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Queues appear whenever work arrives faster than a resource can handle it. An operating system may hold print jobs, runnable processes or outgoing network packets until the appropriate resource becomes available.

Programs use the same idea to represent waiting customers or computations that must be performed in sequence. Although the exact scheduling policy may vary, a Queue provides an organised way to hold work until it can be processed.

Think about people travelling on an escalator or customers waiting in a line: those who enter first normally leave first. Cars waiting at a petrol station follow a similar pattern, joining at the rear and moving forward as earlier cars are served.

Uses of Queues

- Queues are versatile data structures with various use cases across different domains. Here are some common scenarios where queues are particularly useful:

1.Task Scheduling:

- **Operating Systems**: Queues manage processes waiting for CPU time. First-Come, First-Served scheduling processes them in arrival order.
- **Print Spooling**: Print jobs are queued and processed in the order they are submitted.

2.Breadth-First Search (BFS):

- BFS uses a Queue to explore graph nodes level by level.
- Discovered nodes are enqueued and later dequeued for processing.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Task scheduling is one place where arrival order can determine which work receives attention next. First-Come, First-Served scheduling uses a Queue to process waiting tasks in arrival order, although many operating systems also use priorities and more advanced policies.

Print spooling follows a similar pattern by holding submitted jobs until the printer becomes available. Each completed job is removed from the front, allowing the next waiting job to proceed.

Breadth-first search uses a Queue to explore a graph level by level. Nodes are enqueued when discovered and dequeued for examination, ensuring that nearer nodes are processed before those farther from the starting point.

Uses of Queues

3. **Data Buffering**:
   - **Streaming Data**: Queues can buffer data streams (e.g., video/audio streaming), allowing data to be processed as it arrives.
   - **Network Data Packets**: Queues help manage incoming network packets, ensuring they are processed in the order they are received.

4. **Event Management**:
   - In GUI applications, events (like user interactions) can be managed using queues, allowing them to be handled in the order they occur.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Queues are useful when data arrives at a different rate from the component consuming it. A streaming buffer temporarily stores audio or video data, helping playback continue smoothly despite short variations in delivery speed.

Network devices also queue packets while they wait to be processed or transmitted. Some systems use several queues or priority rules, but the basic purpose remains to organise packets awaiting service.

Graphical applications use event queues for clicks, keystrokes and other user interactions. The event loop removes each waiting event and sends it to the appropriate handler, separating the moment an event occurs from the moment it is processed.

Stacks and Queues Example

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

This example brings stacks and queues together so that we can compare their different access orders. By moving Queue elements into a Stack and then transferring them back, the Stack's Last In, First Out behaviour reverses the original Queue order. Follow each value through the two structures and observe how their operations produce the final result.

Stacks and Queues: Working Example 03

Given a Queue data structure, write a function to reverse its elements using a Stack. The original Queue structure should be modified in-place. You can only use standard Queue operations (enqueue, dequeue) and Stack operations (push, pop).

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The question gives us an existing Queue and asks us to reverse the order of its elements. For example, if the Queue contains 10, 20 and 30 from front to rear, it should contain 30, 20 and 10 after the function finishes.

We cannot access the internal nodes or change their links directly. We may interact with the Queue only through enqueue() and dequeue(), and use a temporary Stack only through push() and pop().

The supplied Queue object must hold the reversed sequence at the end; we should not simply return a separate Queue. The task is therefore to use the Stack's LIFO behaviour to reverse the Queue's original FIFO order.

Stacks and Queues: Working Example 03 (cont.)

```
1 class Node:
2    def __init__(self, data):
3        self.data = data
4        self.next = None
5
6 class Stack:
7    def __init__(self):
8        self.top = None
9        self.size = 0
10
11   def peek(self):
12       if self.is_empty():
13           raise IndexError("Peek: Empty stack")
14       return self.top.data
15
16   def push(self, data):
17       new_node = Node(data)
18       new_node.next = self.top
19       self.top = new_node
20       self.size += 1
```

```
21   def pop(self):
22       if self.is_empty():
23           raise IndexError("Pop: Empty stack")
24       popped_node = self.top
25       self.top = self.top.next
26       self.size -= 1
27       return popped_node.data
28
29   def is_empty(self):
30       return self.top is None
31
32   def get_size(self):
33       return self.size
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

This Stack is implemented directly with linked nodes, using top as its only structural reference. The constructor begins with top set to None and size set to zero.

push() creates a node and places it before the current top, while pop() removes and returns the top node's data. peek() inspects that same value without changing the Stack, and the remaining methods report its state.

Because insertion and removal occur at one end, the Stack reverses the order received from the Queue. The last value dequeued and pushed will sit at the top, making it the first value popped during the return transfer.

Stacks and Queues: Working Example 03 (cont.)

```
34 class Queue:
35     def __init__(self):
36         self.front = None
37         self.rear = None
38         self.size = 0
39
40     def isEmpty(self):
41         return self.front is None
42
43     def enqueue(self, data):
44         new_node = Node(data)
45         if self.isEmpty():
46             self.front = new_node
47         else:
48             self.rear.next = new_node
49         self.rear = new_node
50         self.size += 1
```

```
51     def dequeue(self):
52         if self.isEmpty():
53             raise IndexError("Dequeue from empty queue")
54         dequeued_node = self.front
55         self.front = self.front.next
56         if self.front is None:
57             self.rear = None
58         self.size -= 1
59         return dequeued_node.data
60
61     def getFront(self):
62         if self.isEmpty():
63             raise IndexError("Peek from empty queue")
64         return self.front.data
65
66     def getSize(self):
67         return self.size
68
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

The Queue in this program is the direct linked-node implementation we examined earlier. It maintains front, rear and size, with insertion at the rear and removal from the front.

For the reversal algorithm, dequeue() repeatedly removes the current front value and returns it to the caller. When the final node is removed, both front and rear become None, leaving the Queue empty.

Each returned value is pushed onto the Stack immediately, so the data is preserved while its order changes. The algorithm uses only the public methods, which means it does not depend on how either structure manages its internal nodes.

Stacks and Queues: Working Example 03 (cont.)

```
69      def printQueue(self):
70          if self.isEmpty():
71                print("Queue is empty")
72                return
73          current = self.front
74          while current is not None:
75                print(current.data, end=" ")
76                current = current.next
77          print() # New Line
78
79  def reverse_queue(queue):
80      stack = Stack()
81
82      # Push all elements from queue to stack
83      while not queue.isEmpty():
84          stack.push(queue.dequeue())
85
86      # Pop from stack and enqueue back to queue
87      while not stack.is_empty():
88          queue.enqueue(stack.pop())
89
90      return queue
```

```
91  if __name__ == "__main__":
92      queue = Queue()
93
94      # Enqueue three values
95      queue.enqueue(10)
96      queue.enqueue(20)
97      queue.enqueue(30)
98
99      # Print original queue
100     print("Original Queue:", end=" ")
101     queue.printQueue()     # Print: 10 20 30
102
103     # Reverse the queue
104     queue = reverse_queue(queue)
105
106     # Print reversed queue
107     print("Reversed Queue:", end=" ")
108     queue.printQueue()     # Print: 30 20 10
```

rear

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

printQueue() traverses the linked nodes only to display their values, so it does not change the Queue. The reversal begins by creating a temporary Stack to hold the values during the transfer.

The first loop repeatedly calls dequeue() and pushes each returned value onto the Stack. Once the Queue is empty, the second loop pops the values in LIFO order and enqueues them back at the rear.

For the values 10, 20 and 30, the original output is 10, 20, 30, while the reversed output is 30, 20, 10. Every element is transferred from the Queue to the Stack and then back to the Queue, which explains why the complete sequence is reversed.

Stacks and Queues: Working Example 04

Given a Queue data structure, write a function to reverse its elements using a Stack. The original Queue structure should be modified in-place. You can only use standard Queue operations (enqueue, dequeue) and Stack operations (push, pop).



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We are solving the same reversal problem again, but this time both the Stack and Queue will use a shared LinkedList class for storage. The required result and the permitted operations remain exactly the same.

The Queue must still be reversed through enqueue() and dequeue(), while the temporary Stack is accessed through push() and pop(). We will not manipulate the LinkedList nodes directly inside the reversal function.

Only the internal implementation of the two abstract data types has changed. This allows us to see whether the same algorithm works independently of how the Stack and Queue store their elements.

Stacks and Queues: Working Example 04 (cont.)

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
        self.tail = None

    def find_node(self, index):
        if index < 0 or index >= self.size:
            return None
        temp = self.head
        for _ in range(index):
            temp = temp.next
        return temp

    def remove_node(self, index):
        if index < 0 or index >= self.size:
            return -1
        if index == 0:
            self.head = self.head.next
            if self.size == 1:
                self.tail = None
        else:
            prev = self.find_node(index - 1)
            prev.next = prev.next.next
            if index == self.size - 1:
                self.tail = prev
        self.size -= 1
        return 0
```

```python
    def insert_node(self, index, value):
        if index < 0 or index > self.size:
            return -1
        new_node = Node(value)
        if index == 0:
            new_node.next = self.head
            self.head = new_node
            if self.size == 0:
                self.tail = new_node
        elif index == self.size:
            self.tail.next = new_node
            self.tail = new_node
        else:
            prev = self.find_node(index - 1)
            new_node.next = prev.next
            prev.next = new_node
        self.size += 1
        return 0
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

This LinkedList provides the shared storage used by both adapter classes. It maintains head, tail and size, while its insertion and removal methods support different positions.

The Stack will insert and remove at index zero because its top corresponds to the list's head. The Queue will insert at index size and remove index zero, mapping its rear to the tail and its front to the head.

The LinkedList also handles boundary cases, such as creating the first node or removing the final one. Keeping head and tail accurate allows the Stack and Queue to perform their required end operations without unnecessary traversal.

Stacks and Queues: Working Example 04 (cont.)

```
52 class Stack:
53     def __init__(self):
54         self.ll = LinkedList()
55
56     def push(self, data):
57         self.ll.insert_node(0, data)
58
59     def pop(self):
60         if self.is_empty():
61             raise IndexError("Empty Stack")
62         data = self.ll.head.data
63         self.ll.remove_node(0)
64         return data
65
66     def peek(self):
67         if self.is_empty():
68             raise IndexError("Empty Stack")
69         return self.ll.head.data
70
71     def is_empty(self):
72         return self.ll.size == 0
73
74     def get_size(self):
75         return self.ll.size
```

```
76 class Queue:
77     def __init__(self):
78         self.ll = LinkedList()
79
80     def enqueue(self, data):
81         self.ll.insert_node(self.ll.size, data)
82
83     def dequeue(self):
84         if self.isEmpty():
84             raise IndexError("Dequeue from empty queue")
86         data = self.ll.head.data
87         self.ll.remove_node(0)
88         return data
89
90     def getFront(self):
91         if self.isEmpty():
92             raise IndexError("Peek from empty queue")
93         return self.ll.head.data
94
95     def getSize(self):
96         return self.ll.size
97
98     def isEmpty(self):
99         return self.ll.size == 0
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

These two adapter classes use the same LinkedList but impose different access rules. The Stack treats the list's head as its top, so both push() and pop() operate at index zero.

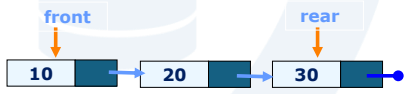
The Queue also removes from index zero, but enqueue() inserts at index size, placing new elements at the tail. This difference produces FIFO behaviour for the Queue and LIFO behaviour for the Stack.

Both classes inspect head when reading the next available value and use the LinkedList's size to report their state. The comparison shows that shared storage can support different abstract data types when each interface controls how elements are accessed.

Stacks and Queues: Working Example 04 (cont.)

```
100     def printQueue(self):
101         if self.isEmpty():
102           print("Queue is empty")
103           return
104         current = self.ll.head
105         while current:
106           print(current.data, end=" ")
107           current = current.next
108         print() # New Line
109
110 def reverse_queue(queue):
111     stack = Stack()
112
113     # Push all elements from queue to stack
114     while not queue.isEmpty():
115         stack.push(queue.dequeue())
116
117     # Pop from stack and enqueue back to queue
118     while not stack.is_empty():
119         queue.enqueue(stack.pop())
120
121     return queue
122
123
```

```
124 if __name__ == "__main__":
125     queue = Queue()
126
127     # Enqueue three values
128     queue.enqueue(10)
129     queue.enqueue(20)
130     queue.enqueue(30)
131
132     # Print original queue
133     print("Original Queue:", end=" ")
134     queue.printQueue()     # Print: 10 20 30
135
136     # Reverse the queue
137     queue = reverse_queue(queue)
138
139     # Print reversed queue
140     print("Reversed Queue:", end=" ")
141     queue.printQueue()     # Print: 30 20 10
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

At first glance, this program looks almost identical to the earlier reversal example. That is exactly what abstraction should give us: reverse_queue() still works only through public Stack and Queue operations.

The visible implementation difference appears in printQueue(), which begins at self.ll.head because the Queue now delegates storage to a LinkedList. The transfer loops require no changes, and the output again changes from 10, 20, 30 to 30, 20, 10.

This confirms that the algorithm depends on FIFO and LIFO behaviour rather than a particular node representation. The internal storage could change again without affecting the reversal, provided both interfaces preserve the same contracts.

Conclusion

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

That brings us to the final part of the lecture. We have explored Queue behaviour through several implementations and examples, so let’s finish by drawing together the main ideas you should take away from this session.

Conclusion

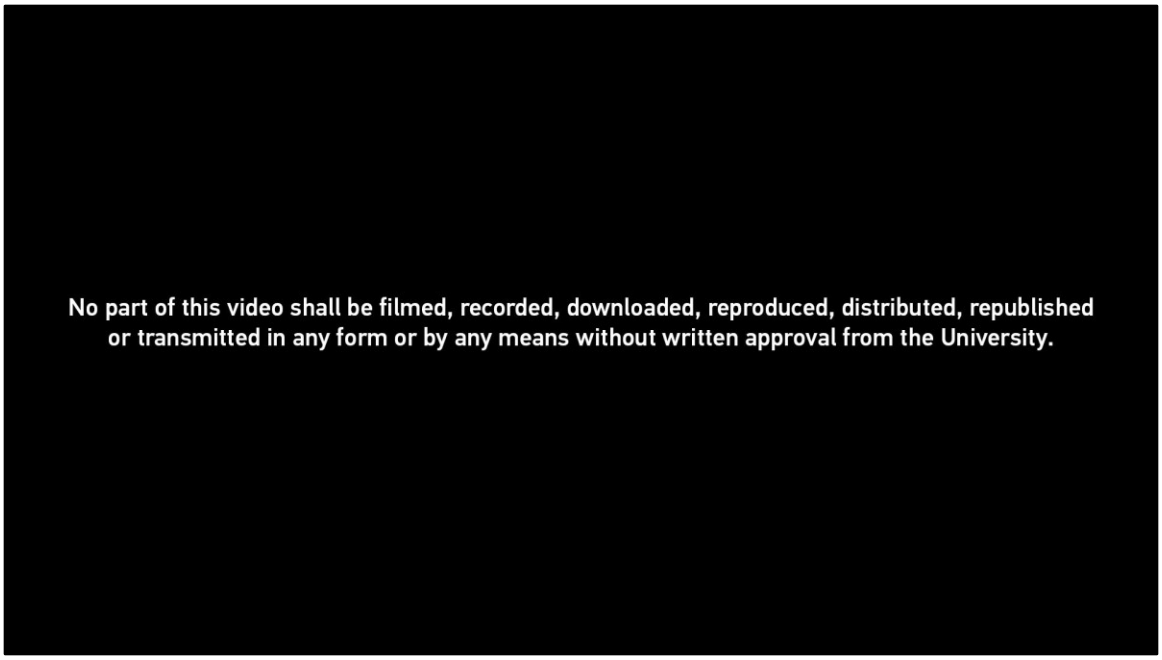
- A **Queue** is a linear data structure that follows the **FIFO** principle.
- Elements are **added at the rear** and **removed from the front**.
- Core operations include **enqueue(), dequeue(), getFront(), isEmpty(), and getSize()**.
- A Queue can be implemented directly using **linked nodes** or through a **LinkedList class**.
- Maintaining front and rear references avoids unnecessary traversal of the linked nodes.
- **enqueue()** and **dequeue()** update the Queue while preserving FIFO order.
- Queues are useful for managing data and tasks that must be processed in **arrival order**.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

A Queue is defined by its First In, First Out behaviour: new elements join at the rear, while removal occurs from the front. This controlled access preserves the order in which elements arrive.

Maintaining references to both ends allows enqueue() and dequeue() to update a linked Queue without traversing all its nodes. Supporting methods then inspect the front, test emptiness and report the size without altering the stored sequence.

We implemented this behaviour both directly with linked nodes and through a reusable LinkedList class. Whichever representation is chosen, the Queue must preserve FIFO order for applications such as scheduling, buffering, event handling and breadth-first search.


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.